



Tecnológico de Monterrey

Campus Puebla

Nombre del trabajo:

Evidencia 1 | Resumen y Reflexión

Curso:

Programación de estructuras de datos y algoritmos fundamentales (TC1031.850)

Estudiante:

Farid Gabriel Velasco Martínez

Matrícula:

A01736669

Docente:

Eduardo Arturo Rodríguez Tello

Fecha de entrega:

09 de junio

Gracias profesor por este semestre de aprendizaje, la clase fue interesante y divertida, aprendí y entendí muy bien todos los temas del curso. Le agradezco por su flexibilidad, disponibilidad y apoyo que me brindó.

Índice:

Introducción	4
Análisis	4
Propuestas de mejora	6
Conclusión:	7

Introducción

Durante esta unidad de formación abordamos una situación problema basada en **ataques cibernéticos** mediante botnets, especialmente en escenarios de ataques **DDOS**. A través de cinco actividades integradoras, analizamos diferentes aspectos del problema mediante el diseño e implementación de algoritmos y estructuras de datos eficientes. Este enfoque práctico nos permitió aplicar de forma directa conocimientos fundamentales de programación, análisis computacional, modelado de estructuras y toma de decisiones algorítmicas, consolidando nuestras competencias en el desarrollo de soluciones reales dentro del ámbito de la ingeniería computacional.

Análisis

A lo largo del curso, desarrollamos diversas estructuras: vectores, listas doblemente ligadas, heaps, grafos y tablas hash. En la primera actividad, optimizamos el ordenamiento y búsqueda sobre vectores de registros usando un enfoque híbrido "Two Step Sort" que combinó Counting Sort y Quicksort, junto con Binary Search y Two Pointers para distintas búsquedas. La eficiencia lograda demostró cómo un análisis profundo permite adaptar algoritmos según las propiedades del conjunto de datos.

En la segunda actividad implementamos listas doblemente ligadas, explorando su eficiencia espacial para grandes volúmenes de datos. Se empleó Merge Sort iterativo y se adaptó Binary Search, aunque con una degradación temporal esperada. La experiencia reflejó el impacto de las decisiones de diseño sobre la complejidad computacional y la necesidad de evaluar cuidadosamente el tipo de estructura usada.

Posteriormente, en la tercera actividad jerarquizamos la información usando un heap, lo que permitió almacenar y analizar registros con operaciones eficientes ($O(\log n)$ en inserción y borrado, $O(1)$ en peeks). Esta estructura demostró ser ideal cuando se necesita acceder al elemento más prioritario rápidamente, reforzando su aplicabilidad en entornos de monitoreo de amenazas.

En la cuarta actividad desarrollamos un grafo dirigido y ponderado para representar las conexiones entre IPs. La combinación con un Binary Heap permitió implementar eficientemente el algoritmo de Dijkstra, optimizando el cálculo de caminos más cortos y detectando el camino de mayor esfuerzo. Esta abstracción de red permitió identificar nodos clave como el bot master, ejemplificando el poder analítico de los grafos en ciberseguridad.

En la quinta actividad integramos tablas hash con quadratic probing. Gracias a su eficiencia temporal ($O(1)$ promedio), se optimizó el almacenamiento y consulta de datos críticos, y se evidenció el impacto del factor de carga sobre el número de colisiones. Experimentamos con gráficas y rehashing, lo que consolidó nuestra comprensión sobre la relación entre diseño estructural y rendimiento real.

No se puede juzgar directamente la implementación de las listas doblemente ligadas junto con los grafos ya que cumplen propósitos diferentes, sin embargo la estructura de datos más versátil, con la que se pudo llegar a conclusiones más cruciales y abstractas fueron los grafos, estos representan las interconexiones entre diferentes elementos o nodos, en este caso las diferentes IPs que se conectaban entre sí y de esta forma, sumado a la complejidad del algoritmo Dijkstra $O((V+E) \log V)$ podemos encontrar caminos y conexiones entre nodos de manera eficiente.

De forma general, integrar estructuras como grafos, heaps y tablas hash resulta ser el enfoque más balanceado para abordar problemas de ciberseguridad. Mientras los grafos permiten abstraer las relaciones complejas, los heaps optimizan la selección de nodos y las tablas hash facilitan acceso instantáneo a información clave. Esta combinación refleja un enfoque de diseño riguroso y eficiente.

La reflexión final que se desprende es que no existe una estructura superior en todos los casos. El valor está en saber elegir y combinar estructuras según el contexto, comprendiendo sus fortalezas y limitaciones. Esta habilidad de análisis y selección diferenciada es esencial para el trabajo ingenieril.

En este proceso, logré desarrollar de forma destacada tres competencias clave. Primero, **la evaluación del problema (SICT0301)**, al analizar la estructura más adecuada para cada actividad integradora, considerando su complejidad computacional y el comportamiento del conjunto de datos. Segundo, **la toma de decisiones (SICT0302)**, al seleccionar e implementar algoritmos como Dijkstra, MergeSort iterativo o técnicas de quadratic probing bajo condiciones específicas de eficiencia y escalabilidad. Y tercero, **la implementación de acciones (SICT0303)**, al diseñar soluciones funcionales en C++ que respondieran a los requerimientos de la situación problema, con una estructura modular, uso de clases, y métodos con fundamentos algorítmicos sólidos.

Propuestas de mejora

Hablando específicamente de los grafos, la implementación del Grafo Dirigido Ponderado no fue del todo convencional, se intentó reducir la complejidad espacial al definir varias Tablas Hash, una específica para la conversión entre dato T a puntero *T y usar los punteros como llaves en otro mapa que almacenara la lista de adyacencia, esto implicó que también se tuviera un vector o lista de llaves T. Esto mantenía la complejidad espacial en

$O(n^2)$, sin embargo incluyó carga computacional innecesaria en la implementación de los algoritmos de grafos, por lo que sería mejor simplificarlo a la pura lista de adyacencia.

Por otro lado, se intentó apegarse al estándar de las implementaciones de heaps, listas o vectores de la STL y se tuvo en cuenta un enfoque general muy general para que el uso de las clases y estructuras implementadas fuera versátil para la solución de otros problemas con el uso de otro tipo de datos, sin embargo especializar las estructuras de datos a los datos manejados, ya sean registros, IPs, fechas, etc., podría haber sido mejor en cuanto a eficiencia temporal y baja carga computacional

Conclusión

Lo que más disfruté del curso fue su enfoque detallado y la claridad del profesor al explicar los conceptos desde sus fundamentos, acompañado de ejemplos prácticos y código en clase. La situación problema elegida fue muy realista, y las soluciones implementadas reflejan con fidelidad las que se adoptan profesionalmente.

Sin embargo, hubo aspectos que me costaron, como el análisis matemático formal de la complejidad temporal, sobre todo en algoritmos recursivos, y me hubiera gustado una exploración más profunda del desarrollo de ciertos algoritmos desde su concepción.

El mayor aprendizaje fue entender que la programación con enfoque ingenieril requiere integrar varias disciplinas: matemáticas, lógica, resolución de problemas y comunicación efectiva. Esta visión amplia me ha ayudado a valorar la importancia del pensamiento crítico y la interdisciplinariedad.

Me llevo una base sólida en C++ y estructuras de datos aplicables a problemas reales, lo cual sin duda será de utilidad en futuras unidades y en mi desarrollo profesional. Además, aprendí a trabajar en equipo reconociendo fortalezas individuales y manteniendo la motivación incluso en momentos desafiantes, aspectos clave en cualquier proyecto real.

Este curso ha sido una experiencia enriquecedora que me ha retado y fortalecido como estudiante de ingeniería.